

Cheat Sheet — Guía de estudio pre-parcial

Contents

Cheat Sheet — Guía de estudio pre-parcial	1
§0 — Metodología del parcial oral	1
§1 — Hosting y data residency	2
§2 — Escalado horizontal: instancias + Load Balancer	2
§3 — Persistencia	3
§4 — Frontend: Webapp vs SPA	5
§5 — Mensajería asincrónica: Queues	5
§6 — Seguridad de conexiones	6
§7 — Autenticación	7
§8 — Protección de datos: encryption	7
§9 — Mapeo problema → solución por atributo de calidad	8
§10 — Tradeoffs canónicos	11
§11 — Anti-patrones que dan 2 automático	12
§12 — Runbook de 7 pasos para el parcial bajo presión	12
§13 — Lo que esta cheat sheet NO cubre (pero podría caer)	13
§14 — Cierre: la frase que la cátedra quiere oír	13
Pregunta a profundizar	13

Cheat Sheet — Guía de estudio pre-parcial

Cómo usar este documento. Está pensado para abrirlo **la noche anterior** al parcial y barrerlo de arriba abajo. La sección §0 te recuerda **cómo responder**. Las §1–§8 son el **bucket de soluciones** con definiciones expandidas. La §9 es el **mapa problema→solución** por atributo. La §10 son **tradeoffs canónicos**. La §11 son los **anti-patrones a no caer**. La §12 es un **runbook de 7 pasos** para resolver el caso bajo presión.

§0 — Metodología del parcial oral

Antes de cualquier mecanismo, **el método** ([[Clase 8 — Consultas pre-parcial]]):

Una respuesta de parcial mínimamente aprobada tiene cuatro cosas: 1. **2 tradeoffs explícitos** (Atributo A vs Atributo B, con la decisión que los resuelve). 2. **1 diagrama de componentes** (entidades del sistema + sistemas externos + protocolos). 3. **Componentes nombrados** (cada caja del diagrama es un mecanismo del bucket). 4. **Escenarios cuantificados** (no “rápido”: “ $\leq 200\text{ ms}$ al p95 para 10k req/s”).

El método ADD aplicado al parcial (15-25 min)

1. **Lee el caso 2 veces.** Subrayá: stakeholders, restricciones, volumen, regulación, criticidad.
2. **Identificá top 4 atributos de calidad.** En el 80% de los parciales son alguna combinación de **Security, Availability, Scalability, Performance**, más uno de **Interoperability / Fault Tolerance / Modifiability** según el dominio.

3. **Ordenalos por prioridad** según el caso. (Banco→Security #1; Streaming→Performance #1; IoT→Scalability #1; Salud→Reliability #1.)
4. **Itera por atributo**: para cada uno, elegí mecanismo del bucket, dibujalo en el diagrama, escribí el escenario.
5. **Identificá 2 tradeoffs** que surgen del diseño y resolvelos explícitamente.
6. **Revisá la lista de anti-patrones** (§11). Si caíste en uno, corregí antes de entregar.

Estructura recomendada del escrito

1. Stakeholders y restricciones (5 líneas).
2. Top 4 atributos de calidad – priorizados y JUSTIFICADOS por el caso.
3. Escenarios cuantificados (1 por atributo top).
4. Diagrama de componentes (con sistemas externos).
5. Iteración por atributo: qué mecanismos eligo y por qué.
6. Tradeoffs (mínimo 2).
7. Riesgos residuales + qué validaría con un prototipo.

(Estructura derivada del esqueleto ATAM aplicado al ejercicio del [[Caso Compraventa de Acciones]] en [[Clase 7 – Caso Compraventa de Acciones]].)

§1 – Hosting y data residency

Tres opciones canónicas, ordenadas de menor a mayor ownership:

Opción	Ventajas	Contras	Cuándo usar
Cloud (AWS, GCP, Azure)	Barato a baja escala; expansión geográfica trivial; alta disponibilidad gestionada; cortes/desastres mitigados por el provider	Pierdes ownership; data residency puede violar regulación; vendor lock-in; cara a alta escala	Default para startup y pyme; sistemas geográficamente distribuidos
On Premise (datacenter compartido o edificio propio)	Ownership total; cumplimiento de normativa local más simple; latencia controlada a usuarios cercanos	Capex alto; el equipo debe operar el datacenter; cortes/desastres dependen de vos	Sistemas regulados (banca, salud, gobierno), gran escala estable
Sala Cofre	Restricción física por candado físico; máximo ownership y máxima protección frente a insider physical attacks	Costo enorme; latencia depende de ubicación; redundancia/HA cuesta más	Datos extremadamente sensibles (defensa, voto, datos médicos críticos)

Criterio de elección (3 ejes): *data residency* (regulación: ¿dónde tiene que vivir físicamente el dato?), *costos* (capex vs opex; quién paga la HA), *latencia* (dónde están los usuarios). Ver [[Criterios de hosting y data residency]].

Trampa común: “lo pongo en AWS porque es lo más fácil” → fail si el caso menciona datos en suelo argentino, regulación PDP, banca local.

§2 – Escalado horizontal: instancias + Load Balancer

Mecanismo: correr N réplicas del backend detrás de un Load Balancer (LB).

Ventajas: - Escalado horizontal bajo demanda (autoscaling con AWS ASG, GCP MIG, Kubernetes HPA). - Disponibilidad: una réplica cae → LB la saca del pool → cliente no nota. - Paralelismo: N CPUs procesan en paralelo.

Riesgos: - **El LB es nuevo SPOF.** Mitigación: LB del cloud provider (multi-AZ por default), o dos LBs en activo-pasivo con failover DNS / Anycast. - **Estado en memoria del backend** rompe si no es sticky (sesiones, web-sockets). Solución: stateless backend + estado en cache/BD compartida, o sticky sessions con session affinity.

Patrón canónico en el parcial: SPA → CDN → WAF → LB → N instancias backend → BD primaria + N réplicas + caché.

Anti-patrón mortal: mencionar “escalo horizontal” sin dibujar un LB. Ver [[Anti-patrones de parcial#“Escalamiento horizontal sin Load Balancer”|Anti-patrón #2]].

§3 — Persistencia

Catálogo completo en [[Persistencia]]. Para parcial, alcanza con dominar 6 opciones:

3.1 — SQL OLTP (transaccional)

Qué es: [[Bases de datos relacionales|RDBMS]] (Postgres, MySQL, Oracle, SQL Server) con ACID, normalización, álgebra relacional.

Cuándo: operación transaccional, escritura concurrente, integridad referencial, queries con joins y agregaciones.

En el [[Teorema CAP]]: CA (sacrificás P bajo partición).

Escala: vertical (más CPU/RAM) natural; horizontal a fuerza de sharding o read replicas.

3.2 — SQL OLAP (analítico)

Qué es: BDs columnares para reporting (Snowflake, BigQuery, Redshift, ClickHouse). Datos desnormalizados, agregaciones precalculadas, queries pesadas.

Cuándo: dashboards, reportes históricos, BI, ML feature engineering.

Cómo poblarlas: ETL/ELT/CDC desde OLTP. Ver [[OLAP y ETL]].

Anti-patrón mortal: usar OLAP para transaccional. Ver [[Anti-patrones de parcial#“Usar bases OLAP para transaccional”|Anti-patrón #5]].

3.3 — Caché

Qué es: BD volátil en memoria (Redis, Memcached, Hazelcast). Acceso sub-ms.

Cuándo: lecturas repetidas, datos derivados costosos, sesiones, resultados de queries pesadas, rate limiting.

Mínimo a documentar en el parcial: - **Qué se cachea** (datos específicos, no “todo”). - **TTL o política de invalidación** (cache-aside con TTL=60s; write-through; refresh-ahead). - **Política de coherencia** y qué pasa con stale reads.

Datos que NO se cachean: saldos bancarios, ACLs/permisos, precios en subasta, datos médicos críticos. Ver [[Anti-patrones de parcial#“Cachear datos que no deberían ser cacheados”|Anti-patrón #3]].

3.4 — Cassandra (columnar distribuida)

Qué es: [[Bases no relacionales|NoSQL columnar]] (Cassandra, ScyllaDB, HBase). Corre en clúster con protocolo gossip; sin nodo coordinador único.

En CAP: AP (alta disponibilidad y tolerancia a partición; consistencia eventual o configurable por query con quorum).

Cuándo: **escrituras masivas** (logs, telemetría, eventos), datasets enormes, geo-distribución, time-series, dispuesto a tolerar lectura eventual.

Cuándo NO: transacciones bancarias, integridad referencial, joins complejos.

3.5 — MongoDB (documental)

Qué es: BD orientada a documentos JSON/BSON. Schema flexible, sharding nativo, réplicas.

En CAP: CP por default (con read concern majority) — sacrifica disponibilidad bajo partición. Configurable.

Cuándo: schema flexible (multi-tenant con campos custom, catálogos heterogéneos), alto volumen de escrituras con sharding por key, datos jerárquicos naturales (perfil de usuario con embebidos).

Cuándo NO: queries analíticas multi-tabla, integridad referencial crítica, transacciones multi-documento masivas.

3.6 — BD de objetos

Qué es: OODBMS (Db4o, ObjectStore, Versant). Persisten grafos de objetos del lenguaje sin mapeo. Ver [[Bases de datos de objetos]].

Cuándo: nicho — modelos con herencia profunda y polimorfismo dominante, donde el ORM se vuelve doloroso.

Realidad: raramente la respuesta correcta en 2026. Default razonable: RDBMS + ORM (Active Record o Data Mapper). Ver [[ORM e impedancia objeto-relacional]].

3.7 — Esquemas de replicación

Catálogo en [[Replicación de BD]]:

Esquema	Escala	Trade-off	Cuándo
Primario-Secundario (Master-Replica)	Lecturas	Las réplicas leen datos <i>eventualmente consistentes</i> ; failover requiere promoción de réplica	Default: 90% de casos donde lecturas » escrituras
Primario-Primario (Multi-Master)	Escrituras	Conflictos de concurrencia (last-write-wins, CRDTs, merge manual); performance peor por sincronización	Sólo cuando las escrituras saturan el primario y se acepta complejidad

Tip de parcial: si el caso dice “muchas lecturas” → P-S. Si dice “muchas escrituras” → sharding antes que P-P (P-P es la última opción).

3.8 — Sharding

Qué es: partir el dataset en N particiones disjuntas, cada una en un nodo distinto. Escalamiento horizontal de escrituras. Ver [[Sharding]].

Decisión clave: la **shard key**. Buena shard key = distribución uniforme + queries del happy path resueltas en una sola partición.

Ejemplos canónicos: - Twitter: shard por user_id (timelines viven con el usuario). - E-commerce: shard por order_id (orders) y customer_id (customers). - Geo: shard por país/región.

Trampa: shard por timestamp → hotspot en la partición actual.

§4 — Frontend: Webapp vs SPA

	Webapp (SSR)	SPA (CSR)
Renderizado	Server-Side Rendering — el server arma el HTML completo en cada request	Client-Side Rendering — el navegador ejecuta JS y arma la UI
Network	HTML completo en cada navegación	HTML inicial + JSON en cada interacción
Performance percibida	Primer paint rápido, navegación lenta	Primer paint lento, navegación instantánea
SEO	□ excelente (Google ve HTML real)	□ requiere SSR/prerender (Next.js, Nuxt)
Carga del server	Alta (procesa visual)	Baja (sirve JSON)
Carga del cliente	Baja	Alta (necesita CPU del usuario)
Offline	Imposible	Posible con service workers
Cuándo	Sitios con SEO crítico (e-commerce público, blogs, marketing)	Aplicaciones interactivas (dashboards, editores, redes sociales internas)

Tip de parcial: si el caso menciona “*que aparezca en Google*” → Webapp o SPA con SSR. Si menciona “*interactividad rica*” o “*realtime*” → SPA.

Híbridos modernos: Next.js / Nuxt / SvelteKit → SSR + hidratación + SPA — lo mejor de los dos mundos. En el parcial podés justificar esta elección.

Webhooks no funcionan con SPA pura (no hay endpoint server donde lleguen). Requieren backend que reciba y empuje al cliente vía WebSocket/SSE.

§5 — Mensajería asincrónica: Queues

Qué es: cola de mensajes (RabbitMQ, Kafka, SQS, Pub/Sub) que desacopla **productor de consumidor** en el tiempo.

Anti-patrón mortal: usar la cola como **request-response sincrónico** (el productor espera la respuesta). Ver [[Anti-patrones de parcial#“Poner colas y esperar que sea request-response”|Anti-patrón #1]].

En el parcial no nombrés “Kafka” — nombrá los requisitos. La cátedra valora *especificar las características que necesita una cola genérica*:

Features a especificar en el parcial

1. **Corre en clúster** — replicación entre nodos, no SPOF.
2. **Persistencia** — los mensajes sobreviven a reinicio del clúster.
3. **TTL** — los mensajes expiran (limpieza automática).
4. **Garantías de entrega** — elegir explícitamente:
 - **At-most-once:** fire-and-forget; aceptable para logs y métricas.
 - **At-least-once:** el más común; *exige consumidores idempotentes* porque puede haber duplicados.
 - **Exactly-once:** coordina transacciones cola↔consumidor; más caro; necesario para movimientos de dinero, contabilidad.
5. **Ordenamiento** — por partición o global; típicamente *por clave* (e.g., todos los mensajes de un usuario en orden, sin orden global).
6. **Dead Letter Queue (DLQ)** — mensajes que fallan N veces consecutivas se aíslan para análisis manual; el consumidor no queda en loop infinito.

Cuándo usar cola

- Fire-and-forget (notificaciones, mails, logs).
- Event-driven (un evento dispara N reacciones; ver pub-sub).
- Work queue (procesos batch, ETL, generación de reportes).
- Smoothing de picos (encolar y procesar al ritmo del consumidor).
- Desacople temporal entre microservicios.

Cuándo NO

- Necesitás la respuesta para continuar → HTTP/gRPC.
 - El cliente necesita feedback inmediato → WebSocket / SSE.
-

§6 — Seguridad de conexiones

Catálogo completo en [[Mecanismos de seguridad]]. Aquí, la cheat sheet de qué citar para qué ataque:

6.1 — HTTPS

Qué es: HTTP sobre TLS/SSL. Cifra el canal con criptografía simétrica (sesión) negociada con asimétrica (handshake).

Defiende contra: Man-in-the-Middle (MitM), sniffing pasivo, alteración en tránsito.

Tip de parcial: *toda* comunicación sobre internet es HTTPS. Si no lo decís explícitamente, perdés puntos baratos.

6.2 — JWT (JSON Web Token)

Qué es: token autocontenido firmado digitalmente, en 3 partes Base64Url (header.payload.signature).

Defiende contra: sesiones replicables/falseables (la firma valida autenticidad).

Cuándo: auth entre microservicios; APIs públicas; SSO; stateless authentication (no necesitás guardar sesión en el server).

Cuándo NO: revocación inmediata difícil (el token vive hasta su expiración); refresh tokens y blacklist resuelven a costo de stateful.

6.3 — WAF (Web Application Firewall)

Qué es: firewall de capa 7 (HTTP/HTTPS). Inspecciona requests y bloquea patrones maliciosos.

Defiende contra: SQL injection, XSS, command injection, DDoS de capa 7 (HTTP flood, slowloris).

No defiende contra: DDoS de capas 3-4 (SYN flood, UDP flood) — para eso, anti-DDoS de red (AWS Shield, Cloudflare).

Tip de parcial: ante DDoS, citar **WAF + anti-DDoS de red** (mejor que solo WAF).

6.4 — VPN

Qué es: túnel cifrado sobre red pública. El usuario remoto se ve como conectado a la LAN.

Cuándo: acceso de empleados/operadores a sistemas internos (admin panels, BDs, herramientas internas); conexión segura entre datacenters.

Trampa: VPN ≠ MFA — la VPN protege el canal, no la autenticación. Usá ambos.

6.5 — WebSockets

Qué es: protocolo full-duplex sobre TCP único persistente. Bidireccional, baja latencia, sin overhead HTTP por mensaje.

Cuándo: chat, notificaciones push, dashboards real-time, cotizaciones live, colaboración (Google Docs, Figma).

Trampa de availability: con multi-instancia + LB, la conexión vive en *una* instancia. Si esa instancia cae, el cliente debe reconectar, **pero la sesión WebSocket es por sí misma stateful** — necesitás manejar reconexión + idempotencia + estado externo (Redis pub/sub para reenviar a la instancia correcta).

Tradeoff explícito del cheat sheet: Performance vs Availability — WebSockets ganan performance, pero degradan availability si no manejas failover de conexión.

6.6 — Webhooks

Qué es: HTTP POST que un sistema dispara hacia una URL del consumidor cuando ocurre un evento. Es push, asíncrono, basado en eventos.

Cuándo: integraciones entre sistemas (Stripe → tu backend al cobrar; GitHub → CI al pushear).

No confundir con WebSocket: webhook es server↔server vía HTTP discreto; WebSocket es persistent socket bidireccional, típicamente cliente↔server.

Limitación: no funciona con SPA pura (no hay endpoint server donde recibir).

§7 — Autenticación

7.1 — Niveles

Mecanismo	Definición	Casos
Single Factor	Solo contraseña	Aplicaciones triviales (foros, juegos)
2FA	2 factores de categorías distintas (típicamente <i>algo que sabés</i> + <i>algo que tenés</i> : password + SMS/OTP)	Default razonable: redes sociales, e-commerce, cuentas medianamente sensibles
MFA	2 o más factores de categorías distintas (sabés/tenés/sos): password + token + biometría	Banca, salud, gobierno, sistemas con info personal sensible

Tip de parcial: si el caso menciona robo de credenciales, *insider threat*, o datos sensibles → MFA.

7.2 — Control de acceso

- **ACL (Access Control List)** en el backend: lista de permisos por usuario/grupo sobre cada recurso.
- **RBAC (Role-Based Access Control):** roles agrupan permisos; usuarios tienen roles.
- **ABAC (Attribute-Based):** políticas evaluadas sobre atributos del usuario, recurso y contexto.

Tip de parcial: *ACL en el backend, nunca solo en el frontend* — el frontend valida UX, el backend valida seguridad.

§8 — Protección de datos: encryption

8.1 — Encryption at Rest

Qué es: cifrar los datos almacenados (discos, BDs, backups, object storage). Sin la clave, el dato físico es ilegible.

Defiende contra: robo de disco/server físico, acceso no autorizado al storage, backup leaks.

Trade-off: **overhead de CPU** en cada read/write. Cifrado moderno (AES-NI por hardware) lo hace tolerable pero no gratis.

8.2 — Encryption in Transit

Qué es: cifrar los datos en movimiento por la red. TLS/SSL (HTTPS, WSS, gRPC sobre TLS), SSH, IPsec.

Defiende contra: sniffing, MitM, interceptación en routers/proxies/datacenter.

Trade-off: overhead de handshake + cifrado en cada conexión. Negligible para HTTPS moderno (TLS 1.3 + AES-NI).

Tip de parcial: mencionar **ambos** para datos sensibles — at rest + in transit. Si el caso menciona regulación (PCI-DSS, HIPAA, PDP), son obligatorios.

§9 — Mapeo problema → solución por atributo de calidad

Esta es la **sección de cierre rápido** del cheat sheet, expandida con porqué y caveat.

9.1 — Availability (“el sistema responde cuando lo necesitamos”)

Métrica canónica: % uptime, MTBF y MTTR. Ver [[MTBF y MTTR]] y [[SLA, SLO, SLI]].

Problema / Riesgo	Mecanismo	Caveat
Ataque DDoS	WAF + anti-DDoS de red (capas 3-4)	WAF solo es capa 7; añadir Cloudflare/Shield para capas inferiores
Caída de una instancia backend	Múltiples instancias + Load Balancer	El LB debe ser HA (multi-AZ, dual provider)
Caída de BD relacional	Replicación Primario-Secundario con failover	Failover ≠ instantáneo; promoción de réplica toma segundos
Caída de BD no relacional	Operación en clúster con replicación (Cassandra, Mongo replica set)	Configurar quorum de escritura/lectura según AP vs CP
Corte de internet del datacenter	Múltiples proveedores de internet (multi-homing, BGP)	Costo alto; mitigación parcial: failover DNS
Caída de la cola de mensajes	Cola en clúster con replicación de mensajes	Especificar replication factor ≥ 3
Corte de luz	UPS + generador o doble servicio eléctrico	On-premise; cloud lo abstrae
Caída del activo en HA	Heartbeat entre activo y pasivo + failover automático	Heartbeat con timeout corto (1-3s); split-brain debe estar resuelto

9.2 — Security (“el sistema resiste el ataque adversarial”)

Filosofía: *threshold defense* / *defense in depth* — múltiples capas que un atacante debe atravesar. Ver [[Mecanismos de seguridad]].

Problema / Riesgo	Mecanismo	Caveat
DDoS	WAF + anti-DDoS de red	Ver Availability
Man-in-the-Middle	HTTPS en todas las conexiones	TLS 1.2+, certificados válidos, HSTS

Problema / Riesgo	Mecanismo	Caveat
Insider con acceso privilegiado	VPN + segmentación de red + ACL granular	VPN sola es insuficiente; MFA + auditoría
Robo de credenciales	MFA (mínimo 2FA)	Recovery flow debe ser igualmente seguro
Gestión de accesos	ACL/RBAC/ABAC en el backend	Frontend valida UX, backend valida seguridad
SQL Injection	Sanitización + queries parametrizadas + ORM seguro + WAF	“Sanitizar en la SPA” sirve para UX, pero el backend debe sanitizar también
Acceso malicioso a la BD	Encryption at rest + control de accesos a BD + audit log	Encryption at rest no defiende contra acceso autenticado
Datos extremadamente sensibles	Hosting on-premise / sala cofre	Costo enorme; reservar para casos donde la regulación lo exige
Tampering en tránsito	HTTPS + checksums + firma digital	TLS asegura confidencialidad e integridad de canal

Citación obligatoria: la lista oficial de mecanismos vista en [[Clase 7 — Caso Compraventa de Acciones]] (WAF, MFA, sanitización, encryption at rest, VPN, reverse proxy). Ver [[Mecanismos de seguridad]].

9.3 — Performance (“el sistema responde rápido”)

Métrica canónica: latencia p50/p95/p99, throughput (req/s), tiempo de respuesta percibido.

Problema / Riesgo	Mecanismo	Caveat
Lecturas lentas (BD bottleneck)	Caché en el backend	Documentar qué, TTL, política de invalidación
Demasiadas solicitudes concurrentes	Paralelismo: múltiples instancias + multithreading + async I/O	Cuello de botella se mueve a la BD → escalar también la BD
Reportes en tiempo real	Read replica de la BD primaria	Lag de replicación; ok para reportes operacionales
Reportes históricos/analíticos	OLAP + ETL desde OLTP, ejecutar en ventana de baja demanda	OLTP para operación, OLAP para reportes — <i>nunca al revés</i>
Búsquedas en múltiples fuentes externas	Llamadas paralelas asincrónicas con timeout + circuit breaker	Total = max() en vez de sum() de latencias individuales
Lectura/escritura en tiempo real	WebSocket / Server-Sent Events	Trade-off availability (ver §6.5)
Latencia geográfica para frontend	CDN (CloudFront, Cloudflare, Fastly)	Solo sirve para assets estáticos; el HTML dinámico requiere edge compute
Cómputo CPU-intensive	Off-line precompute + cache de resultados	Ver [[Off-line vs Online computation]]

9.4 — Precision (“los datos numéricos son correctos”)

Atributo crítico en sistemas con sensores, contabilidad, científicos. Ver [[Ejercicio 2013 — Sistema de control de fábrica con sensores serie]].

Problema	Mecanismo	Caveat
Error de punto flotante (IEEE 754)	Aritmética de punto fijo (BigDecimal, decimal128)	Más lenta que float; el dato debe entrar en el rango del fixed
Sensor que puede medir mal	Dos sensores en activo-activo + cross-check + alarma de divergencia	Si difieren más de ϵ , alarma; el más confiable gana o se descarta el dato

Problema	Mecanismo	Caveat
Propagación de errores en cálculos intermedios	Métodos numéricos estables (Kahan summation, escalado, evitar restas de magnitudes similares)	Conocimiento de análisis numérico
Faltan mediciones (sensor offline temporal)	Interpolación (lineal, spline) entre puntos válidos	Solo válido para intervalos cortos; documentar incertidumbre
Mediciones con varianza alta	Intervalos de confianza + ventanas móviles + filtros (Kalman, mediana móvil)	El reporte debe incluir el rango, no solo el valor central
Necesidad de precisión exacta a nivel string	Guardar como string en BD (no como float ni int)	Costo: las operaciones requieren parseo

9.5 — Scalability (“el sistema crece sin romperse”)

Distinguir: escalabilidad vertical (más recursos al mismo nodo) vs horizontal (más nodos). Default moderno: horizontal.

Problema	Mecanismo	Caveat
Picos recurrentes de tráfico	Load Balancer + autoscaling de instancias	Tiempo de cold start; pre-warming para picos predecibles
Muchos accesos a la SPA desde distintas regiones	CDN	Cache invalidation para deploys; configurar TTL por asset
BD se vuelve cuello de botella en escrituras	Sharding (partición por shard key)	Shard key es decisión arquitectónica clave; ver [[Sharding]]
Crecen las lecturas	Read replicas (P-S) + caché	El lag de replicación impone consistencia eventual en las réplicas
Crecen los eventos (logs, telemetría)	Cassandra / time-series DB (InfluxDB, TimescaleDB)	Sacrificio: consistencia eventual
Necesidad de escalar escrituras de forma masiva	Sharding + cola + procesamiento asincrónico + Cassandra	Combina varios mecanismos; ver Twitter ([[Caso Twitter — Big Data en tiempo real]])

9.6 — Interoperability (“el sistema integra con otros”)

Problema	Mecanismo
Dos sistemas con interfaces incompatibles	Patrón Adapter (estructural GoF) — traductor entre interfaz que el cliente espera y la que la clase ofrece
Permitir que terceros integren datos	Exponer API REST/GraphQL solo de lectura, versionada (v1/, v2/), con rate limiting y autenticación (API keys, OAuth)
Integración con sistemas legacy	Anti-corruption layer (DDD) entre el dominio nuevo y el legacy
Múltiples consumidores con distintos formatos	Adapter por consumidor, o API gateway que normaliza

9.7 — Fault Tolerance (“el sistema sigue funcionando ante fallas parciales”)

A diferencia de Availability (binaria: arriba o abajo), Fault Tolerance es la **capacidad de degradar parcialmente** sin caer entero.

Problema	Mecanismo
Falla un componente no crítico (e.g. carga de archivo para reporte)	Graceful degradation — el sistema sigue funcionando con la información disponible y marca el dato faltante
Cae un servicio externo no esencial	Circuit breaker → fallback → degradación de feature
Falla parcial de la BD (1 réplica de N)	Las otras N-1 toman el tráfico; el sistema sigue (eventual consistency aceptada)
Mensaje que falla N veces consecutivas	Dead Letter Queue + alarma + intervención manual

9.8 — Accessibility (“el sistema lo pueden usar todos”)

Mecanismo
Testing con usuarios reales — incluyendo personas con discapacidades
Internacionalización (i18n) y localización (l10n) — strings extraídos, formato de fecha/moneda por locale
Diseños HCI con principios establecidos (Nielsen, WCAG)
Alt text en imágenes para lectores de pantalla
Soporte explícito para screen readers (NVDA, JAWS, VoiceOver) — ARIA roles, semántica HTML
Contraste de color, tamaño de fuente ajustable, navegación por teclado
Subtítulos para video, transcripciones para audio

§10 — Tradeoffs canónicos

Memorizar — los podés citar como justificación de cualquier decisión.

Tradeoff	Mecanismo que lo evidencia	Cómo resolverlo
Availability ↔ Costos	Escalado horizontal (más nodos = más HA = más caro)	Definir SLO realista; HA total cuesta enormemente más que 99.9%
Performance ↔ Security	Encryption at rest (cifrar/descifrar tiene CPU cost)	Aceptar el overhead; cifrado por hardware (AES-NI) lo hace tolerable
Precision ↔ Performance	Punto fijo (más lento que punto flotante)	Aceptar overhead donde la precisión es no negociable (dinero, sensores críticos)
Performance ↔ Availability	WebSockets (conexión persistente en una instancia = no HA por default)	Failover de conexión + estado externo
Consistency ↔ Availability ([[Teorema CAP]])	NoSQL distribuidas (Cassandra=AP; Mongo configurable)	Elegir según dominio: financiero → CP; analytics → AP
Scalability ↔ Consistency	Sharding + read replicas → eventual consistency	Aceptar consistencia eventual donde la app lo tolera
Modifiability ↔ Performance	Microservicios (red entre servicios cuesta)	Modular monolith mientras el equipo sea chico; microservicios cuando duele lo suficiente
Security ↔ Usability	MFA, frecuencia de re-auth, restricciones de password	Diseñar el flow de UX con security in mind, no como afterthought
Time-to-market ↔ Quality	BDUF vs YAGNI vs JEDUF	Default JEDUF — <i>just enough design up front</i> . Ver [[BDUF vs YAGNI vs JEDUF]]

§11 — Anti-patrones que dan 2 automático

Lista mortal. Antes de entregar, releé el diagrama y barré esta lista. Detalle completo en [[Anti-patrones de parcial]].

1. **“Poner una cola y esperar que sea request-response”** — la cola es asincrónica; si esperás respuesta, no ganaste nada.
2. **“Escalado horizontal sin Load Balancer”** — N réplicas sin LB no balancean carga.
3. **“Cachear sin documentar TTL ni política de invalidación”** — el cache es una bomba de stale data.
4. **“Olvidarte sistemas externos”** — el procesador de pago, el carrier, el IdP, el SDK — son SPOFs si no los mencionás.
5. **“Usar OLAP para transaccional”** — Snowflake/BigQuery no es para operación en vivo.

Otros igual de graves: - Microservicios desde día 0 con equipo de 5. - Big bang release sin staging ni canary (lección Healthcare.gov). - Diagrama sin drivers articulados antes. - Escenarios sin números (*“rápido”*, *“seguro”*). - Asumir red perfecta entre componentes distribuidos.

§12 — Runbook de 7 pasos para el parcial bajo presión

Cronómetro mental: ~20-25 min por caso, dependiendo de la duración del examen.

Paso 1 — Leer el caso (2 veces, 2 min)

Subrayar: dominio, usuarios, volumen, regulación, restricciones explícitas, sistemas externos mencionados, criticidad.

Paso 2 — Identificar drivers (3 min)

Listar atributos de calidad relevantes. Priorizar **top 4**. Justificar cada uno con una línea del caso.

Patrón de mapeo dominio → top: - **Banca / Fintech** → Security, Availability, Consistency, Auditability. - **E-commerce** → Scalability, Availability, Performance, Interoperability. - **Streaming / CDN** → Performance, Scalability, Availability, Cost. - **Salud / Telemedicina** → Security, Reliability, Privacy, Interoperability (HL7/FHIR). - **IoT / Smart City** → Scalability, Fault Tolerance, Performance, Availability. - **Gov / Voto** → Security, Auditability, Verifiability, Availability. - **Logística** → Availability, Scalability, Interoperability, Performance.

Paso 3 — Escenarios cuantificados (3 min)

Un escenario por atributo top. Formato:

“Bajo carga normal de N usuarios concurrentes (X req/s), el sistema debe responder al p95 en $\leq Y$ ms y al p99 en $\leq Z$ ms. Bajo pico de M usuarios concurrentes, $p95 \leq Y'$ ms.”

Sin números, no es escenario. Inventá números razonables si el caso no los da, pero *documentá la suposición*.

Paso 4 — Diagrama (5-7 min)

Cajas + flechas. Incluir: - **Usuarios** (con tipo: cliente, operador, admin). - **Frontend** (SPA / Webapp + CDN). - **Edge** (WAF, LB). - **Backend** (N instancias). - **Persistencia** (OLTP + réplicas + caché + OLAP si aplica). - **Cola** (si hay async). - **Sistemas externos** (procesador de pago, IdP, SMS gateway, etc.). - **Protocolos** (HTTPS sobre todo).

Paso 5 — Iteración por atributo (5-7 min)

Para cada uno del top 4: *“Para satisfacer [atributo], introduzco [mecanismo del bucket]. Esto resuelve [escenario] pero introduce [trade-off].”*

Paso 6 — Tradeoffs explícitos (2 min)

Citar mínimo 2 tradeoffs de §10. Decir *cómo* los resolvés en este caso.

Paso 7 — Revisión final (2 min)

- ¿Top 4 atributos están justificados por el caso? (no genéricos)
- ¿Hay escenarios cuantificados? (números, no adjetivos)
- ¿Hay 2 tradeoffs? (no 0)
- ¿Hay un Load Balancer si dije “escalo horizontal”?
- ¿Las colas son asincrónicas (fire-and-forget)?
- ¿La caché tiene TTL/política?
- ¿Aparecen los sistemas externos?
- ¿No estoy usando OLAP para operación?
- ¿Aparece HTTPS en todas las conexiones públicas?

Si todo pasa → entregar.

§13 — Lo que esta cheat sheet NO cubre (pero podría caer)

Repasar también:

- **[[Atributos de calidad]]** — taxonomía completa ISO 25000 (la cheat sheet cubre solo los más comunes; pueden caer Modifiability, Portability, Testability, Manageability).
 - **[[Estilos arquitectónicos]]** — pipes&filters, broker, pub-sub, MVC/MVP/MVVM, event-driven; útil para *nombrar* el estilo en el diagrama.
 - **[[Modelo 4+1]] y [[C4 Model]]** — si el enunciado pide “vistas” o “distintos niveles de abstracción”, mencionar.
 - **[[SAAM]], [[ATAM]], [[Lightweight ATAM]]** — métodos de evaluación; si el enunciado pide “cómo evaluarías” o “riesgos”, citar.
 - **[[Árbol de utilidad]]** — herramienta concreta para conectar atributos abstractos con escenarios priorizados.
 - **[[BDUF vs YAGNI vs JEDUF]]** — postura defendida por la cátedra: JEDUF.
 - **[[Criterios de hosting y data residency]]** — si el caso menciona regulación, datos en suelo argentino, latencia geográfica.
 - **[[Caso Healthcare.gov]]** — lecciones canónicas: identity verifier como SPOF, big bang release, falta de testing de carga.
 - **[[Caso Twitter — Big Data en tiempo real]]** — para casos a escala (timelines, social graphs): [[Partition, Replicate, Index]], [[Off-line vs Online computation]], [[Memory hierarchy]].
 - **[[Caso Compraventa de Acciones]]** — para sistemas financieros: iteración ADD con Security #1, mecanismos de seguridad concretos.
-

§14 — Cierre: la frase que la cátedra quiere oír

“Para este caso, prioricé los atributos [A, B, C, D] porque el enunciado dice [X]. Para A, elijo [mecanismo del bucket] porque [justificación], aceptando el trade-off [T1] que resuelvo con [decisión]. El riesgo residual es [R], que validaré con [prototipo / spike / load test].”

Si esa frase es la **estructura** de cada iteración del parcial, vas bien.

Pregunta a profundizar

¿Cuál es el **límite** del enfoque “bucket de soluciones”? Hipótesis: funciona en parciales tipo cocktail-of-tactics, pero se rompe cuando el caso exige una **decisión estratégica no enumerable** (e.g. “¿escindimos este bounded context en microservicio?”, “¿reescribimos desde cero?”, “¿migramos a otra cloud?”). Para esos casos, el bucket

es insuficiente y hace falta razonamiento estratégico tipo [[Caso Healthcare.gov]] — historia, organización, política, evolución. Si el parcial cae en ese territorio, el cheat sheet ayuda menos.